Problem 1: Matrix Operations using Object-Oriented Programming

Total Marks: 7

Write a C++ program to perform matrix operations using object-oriented programming concepts such as **inheritance**, **operator overloading**, and **runtime polymorphism**.

The program should support three execution modes. The first line of input specifies which part of the program should execute.

- **P1 – Matrix Input and Display (1 Mark)**
- **P2 – Matrix Operations using Operator Overloading (3 Marks)**
- **P3 – Runtime Polymorphism using Inheritance (3 Marks)**

Part 1 – Matrix Input and Display

Create a class **Matrix** with the following members.

Data Members

- **rows** – number of rows
- **cols** – number of columns
- **mat** – a dynamically allocated 2D array to store matrix elements

Member Functions

- Constructor to initialize matrix dimensions and dynamically allocate the 2D array.
- `input()` – reads matrix elements.
- `display()` – prints the matrix.

Task

Read a matrix and display it using the `display()` function.

Input Format

```
P1
r c
Next r lines contain c integers each
```

- r – number of rows
- c – number of columns

Output Format

Display the matrix in row format.

Example

Input

```
P1
2 3
1 2 3
4 5 6
```

Output

```
1 2 3
4 5 6
```

Part 2 – Operator Overloading for Matrix Operations

Overload the following operators for matrix objects.

Operator	Operation
+	Matrix Addition
*	Matrix Multiplication
==	Matrix Equality Check

The operations must be performed using expressions such as:

```
C = A + B
C = A * B
A == B
```

Note: For all operations, the compatibility conditions must be satisfied. If the condition is not satisfied, the program must output: **"incompatible matrices"**.

Input Format

```
P2
r1 c1
Next r1 lines contain c1 integers each
r2 c2
Next r2 lines contain c2 integers each
op
```

Operation Codes

op	Operation
1	Matrix Addition
2	Matrix Multiplication
3	Matrix Equality Check

Output Format

- If $op = 1$ – print the resultant matrix in row format.
- If $op = 2$ – print the resultant matrix in row format.
- If $op = 3$ – print **true** or **false**.

If the compatibility condition is not satisfied, the program must output: **"incompatible matrices"** for all three operations.

Mandatory Requirement

Matrix operations must be performed **only using overloaded operators**. Programs that perform the operations using normal functions instead of operator overloading will not receive marks for Part 2.

Part 3 – Inheritance and Runtime Polymorphism

Extend the **Matrix** class using inheritance.

Base Class

The class **Matrix** must contain a virtual function:

```
virtual void compute();
```

Derived Classes

SquareMatrix

- Inherits from **Matrix**
- Represents matrices where the number of rows equals the number of columns
- Override `compute()` to calculate the **determinant**

DiagonalMatrix

- Inherits from **SquareMatrix**
- Represents a special square matrix in which all elements outside the main diagonal are zero
- Override `compute()` to check whether the matrix is diagonal

Input Format

```
P3
type
n
matrix elements
```

type	Matrix Type
1	SquareMatrix
2	DiagonalMatrix

- n – size of the square matrix
- Next n lines contain n integers each

Output Format

- If type = 1 – print the determinant
- If type = 2 – print whether the matrix is diagonal (**true**/**false**)

Example

Input

```
P3
2
3
1 0 0
0 5 0
0 0 9
```

Output

```
true
```

Mandatory Runtime Polymorphism Requirement

Objects must be created using a base class pointer:

```
Matrix* m;
```

The function `compute()` must be called through the base class pointer to demonstrate **runtime polymorphism**. Programs that fail this criterion will not receive marks for Part 3.